

Supervised Learning: Classification

“Logistic Regression and Discriminant Analysis for categorical outcomes”

Mahdi Shafiee Kamalabad & Maarten Cruyff

1. Logistic Regression

- a probabilistic method for classification

2. Discriminant Analysis

- classification based on class distributions

3. Classification Criteria

- how we evaluate and compare classifiers

Categorical outcomes

We are interested in predicting **categories (classes)** rather than numerical values.

Examples:

- Will a person vote or not, given background characteristics?
- Is a patient healthy or ill, given a set of symptoms?
- Will a person successfully quit smoking?

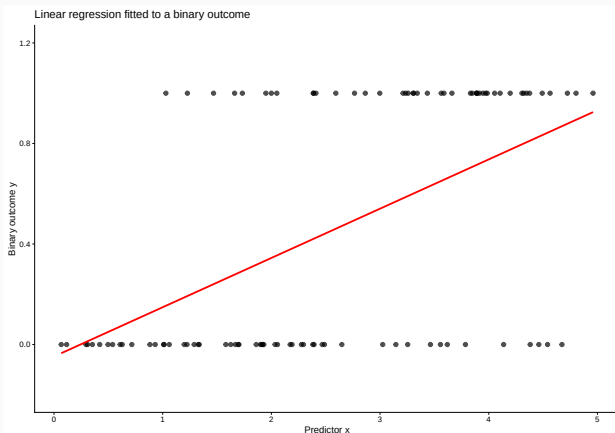
These problems involve predicting a **class**, not a **continuous score**.

Linear regression is not appropriate for this task. But why?

Why not linear regression?

Linear regression is designed for **continuous outcomes**, not categorical/binary classes.

- Predictions are not restricted to valid values (e.g. < 0 or > 1)
- Model assumptions are violated (normal errors, constant variance)
- The fitted values do not naturally correspond to class labels
- A straight-line fit does not respect, the binary nature of the response



**So, what models should we use
instead? Classification**

What is classification?

The outcome variable is **categorical**.

- Goal: predict **class membership** from a set of features

General approach

1. Compute a score or estimate the probability that an observation belongs to each class
2. Assign the observation to the class with the highest score or probability

Common classification models

(roughly ordered by interpretability)

- Logistic regression
- Discriminant analysis
- Trees
- Random Forests, Bagging, Boosting
- Support Vector Machines (SVM)

Score or probability for each class

1. For a given observation with feature vector $\mathbf{x}$, compute a classification score or probability for each class:

$$S_j(\mathbf{x}) \quad \text{or} \quad P(Y = j \mid \mathbf{x})$$

2. Assign the observation to the class with the highest score or probability.

Logistic Regression

Logistic regression: idea

- Logistic regression is used to model the probability of a categorical/binary outcome, e.g. $Y \in \{0, 1\}$.
- It combines a linear model with a transformation that ensures predicted probabilities of classes lie between 0 and 1.

$$P(Y = 1 \mid x) = \frac{e^{\beta_0 + \beta_1 x_1 + \dots}}{1 + e^{\beta_0 + \beta_1 x_1 + \dots}}$$

- How?

Logistic regression: step-by-step

Step 1: Start with a linear predictor

$$\eta = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$$

The linear predictor η can take any real value.

Step 2: Transform it into a probability

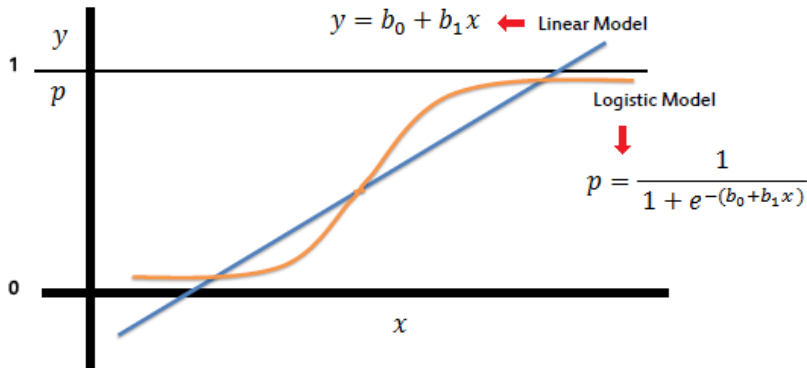
$$P(Y = 1 \mid x) = \frac{1}{1 + e^{-\eta}} = \frac{e^{\eta}}{1 + e^{\eta}}$$

The logistic function maps η into the interval $[0, 1]$, so the result can be interpreted as a probability for a binary outcome:

$$Y \in \{0, 1\}$$

Logistic vs linear regression

- estimates of logistic model in interval (0, 1)
- estimates of linear model are not probabilities



Logistic Regression for Classification in R: `glm()`

Logistic regression is a special case of a **GLM**.

A GLM consists of three main components:

- **Random component:** The distribution of the outcome variable
 - In `'glm()'`: `family = "binomial"` specifies the probability distribution of the response variable, i.e. the random variation in the outcome.

- **Systematic component (linear predictor):**

$$\eta = \beta_0 + \beta_1 x_1 + \dots$$

- In `'glm()'`: Specified by the model formula
- **Link function:** Transforms the probability to a scale where a linear model can be used (η)
 - In `'glm()'`: `link = "logit"` (default for binomial)

Logistic Regression for Classification in R: glm()

Fit the logistic regression model

```
fit_glm <- glm(y ~ x1 + x2 + ...,  
              family = "binomial",  
              link    = "logit",      # default for "binomial"  
              data    = <data>)
```

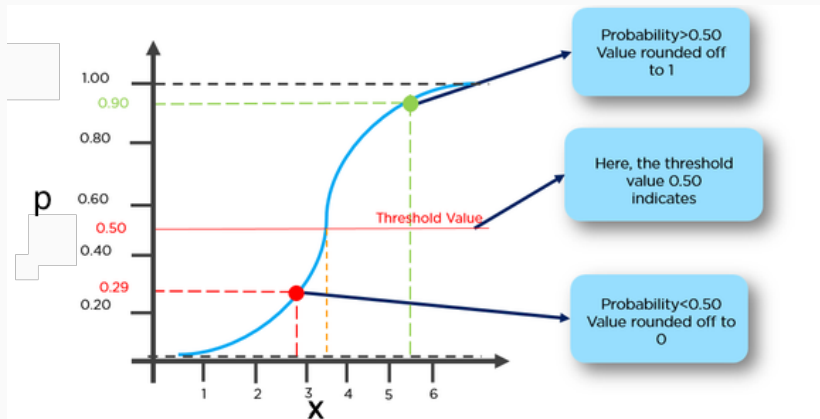
get the predicted probabilities

```
pred_glm <- predict(fit_glm,  
                   data = <data>,  
                   type = "response") # default is "link"
```

Classify as success when larger than threshold

```
class_glm <- factor(pred_glm > 0.5,      # threshold for success  
                   labels = c("success", "failure"))
```

Schematically



Example Pima diabetes data

The Pima dataset contains medical measurements for **532 adult females** of Pima Indian heritage living near Phoenix.

- Outcome variable: type
- Indicates whether a woman has diabetes (Yes / No)
- Diagnosis follows (World Health Organization) WHO criteria

Description: df [532 x 8]							
npreg	glu	bp	skin	bmi	ped	age	type
<int>	<int>	<int>	<int>	<dbl>	<dbl>	<int>	<fctr>
6	148	72	35	33.6	0.627	50	Yes
1	85	66	29	26.6	0.351	31	No
1	89	66	23	28.1	0.167	21	No
3	78	50	32	31.0	0.248	26	Yes
2	197	70	45	30.5	0.158	53	Yes
5	166	72	19	25.8	0.587	51	Yes
0	118	84	47	45.8	0.551	31	Yes
1	103	30	38	43.3	0.183	33	No
3	126	88	41	39.3	0.704	27	No
9	119	80	35	29.0	0.263	29	Yes

Important R convention

When the outcome variable is a factor, the **first level is treated as the reference (failure) class**.

```
levels(Pima$type)
```

```
[1] "No"  "Yes"
```

Fit the model

```
# Fit logistic regression model
```

```
pima_glm <- glm(type ~ .,  
                 family = binomial,  
                 data = Pima.tr)
```

```
# Coefficient estimates
```

```
coef(summary(pima_glm)) %>% round(3)
```

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-9.773	1.770	-5.520	0.000
npreg	0.103	0.065	1.595	0.111
glu	0.032	0.007	4.732	0.000
bp	-0.005	0.019	-0.257	0.797
skin	-0.002	0.022	-0.085	0.932
bmi	0.084	0.043	1.953	0.051
ped	1.820	0.666	2.735	0.006
age	0.041	0.022	1.864	0.062

Interpreting the coefficients

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-9.773	1.770	-5.520	0.000
npreg	0.103	0.065	1.595	0.111
glu	0.032	0.007	4.732	0.000
bp	-0.005	0.019	-0.257	0.797
skin	-0.002	0.022	-0.085	0.932
bmi	0.084	0.043	1.953	0.051
ped	1.820	0.666	2.735	0.006
age	0.041	0.022	1.864	0.062

- Coefficients are on the **log-odds** scale
- A positive coefficient means: increasing the predictor increases the probability of diabetes (holding other variables fixed)
- A negative coefficient means: increasing the predictor decreases the probability of diabetes (holding other variables fixed)

Example

A positive coefficient for `npreg` indicates that women with more pregnancies have a higher probability of diabetes, holding other variables fixed.

The coefficient for `npreg` is

$$\beta_{\text{npreg}} = 0.103$$

Exponentiating gives the odds ratio:

$$e^{0.103} \approx 1.109$$

So, holding all other variables constant, one additional pregnancy multiplies the odds of diabetes by about **1.109**, corresponding to an approximately **10.9% increase in the odds**.

Classification using Logistic Regression

- Logistic regression gives probabilities first. Classification comes from choosing a threshold.

Predicted probabilities and classifications for the first 10 test observations:

```
# Predicted probabilities on the test set
p <- predict(pima_glm, newdata = Pima.te, type = "response")
# Classification using a 0.5 threshold
results <- data.frame(
  probability = round(p, 3),
  classification = factor(p > 0.5, labels = c("no diabetes", "diabetes")),
  observed = Pima.te$type
)
results[1:8, ]
```

	probability	classification	observed
1	0.768	diabetes	Yes
2	0.040	no diabetes	No
3	0.025	no diabetes	No
4	0.041	no diabetes	Yes
5	0.796	diabetes	Yes
6	0.735	diabetes	Yes
7	0.410	no diabetes	Yes
8	0.220	no diabetes	No

Pros and cons of logistic regression

Pros

- Easy to interpret coefficients
- Few assumptions about the distribution of the predictors

Cons

- Unstable estimates with many predictors or rare categories
- Limited flexibility for complex nonlinear patterns

Discriminant Analysis

Motivation

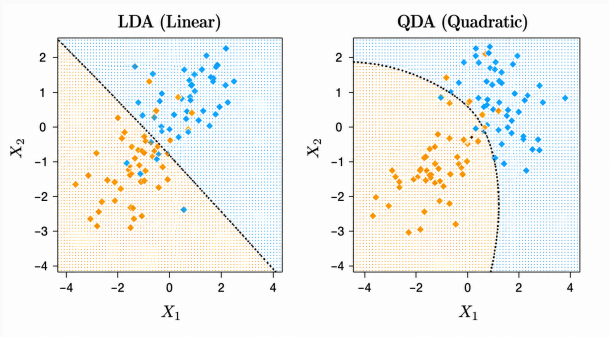
Logistic regression is easy to interpret, but it can be limited when class separation is more complex.

Discriminant analysis provides an alternative approach by explicitly modeling how classes differ.

What is discriminant analysis?

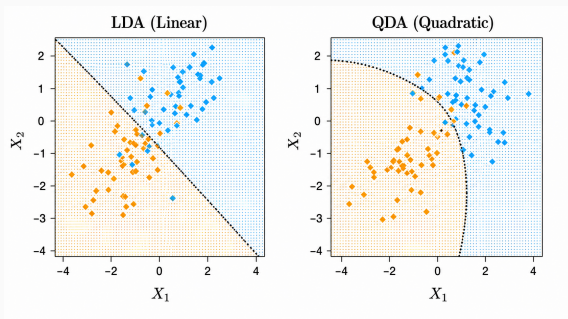
Discriminant analysis is a classification approach that assigns observations to classes by constructing **discriminant functions** that separate the groups.

- Discriminant functions use the predictor variables to score or classify observations.
- The aim is to separate observations from different classes as clearly as possible.



Linear Discriminant Analysis (LDA) and Quadratic Discriminant Analysis (QDA)

- In **Linear Discriminant Analysis (LDA)**, the discriminant functions are linear, giving linear decision boundaries.
- In **Quadratic Discriminant Analysis (QDA)**, the discriminant functions can include quadratic terms, giving curved decision boundaries.



Linear Discriminant Analysis (LDA)

- LDA is a classification method based on **Bayes' rule**.
- It assigns an observation x to the class with the largest posterior probability:

$$P(Y = j \mid X = x) = \frac{\pi_j P(X = x \mid Y = j)}{\sum_{k=1}^J \pi_k P(X = x \mid Y = k)}, \quad j = 1, \dots, J$$

- π_j is the **prior probability** of class j (often estimated by the class proportion)
- $P(X = x \mid Y = j)$ is the **class-conditional density** of the features in class j
- Since the denominator is the same for all classes, LDA only compares:

$$\pi_j P(X = x \mid Y = j)$$

- Taking the log gives the discriminant function:

$$\delta_j(x) = \log \pi_j + \log P(X = x \mid Y = j)$$

Linear Discriminant Analysis (LDA)

- X is multivariate normal within each class:
 - $X | Y = j \sim \mathcal{N}(\mu_j, \Sigma)$
 - X has covariance matrix Σ within each class.
 - All classes share the same covariance matrix Σ
 - The maximum number of discriminant directions is

$$k = \min(\text{\#classes} - 1, \text{\#features})$$

- LDA assigns x to the class with the largest discriminant score:

$$\hat{Y} = \arg \max_j \delta_j(x)$$

- The decision boundary is where two class scores are equal:

$$\delta_j(x) = \delta_k(x)$$

- Because LDA uses the same covariance matrix Σ for all classes, the boundary is linear.

Linear discriminant directions

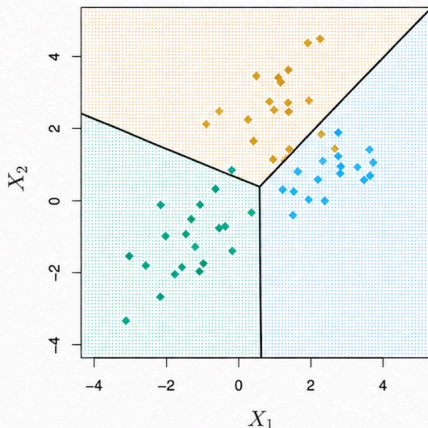
Linear discriminant functions

$$LD_j = c_{1j}X_1 + \cdots + c_{pj}X_p$$

- LD_1 separates the classes best, LD_2 second best, and so on
- LD 's are orthogonal
- They are used to transform the original predictors into new coordinates

Example of LDA with three classes

The solid black lines are the linear decision boundaries estimated by LDA.

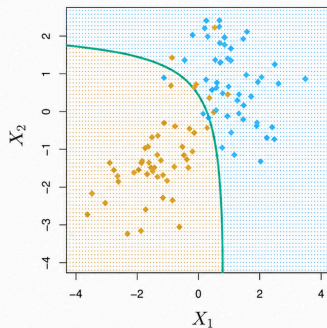
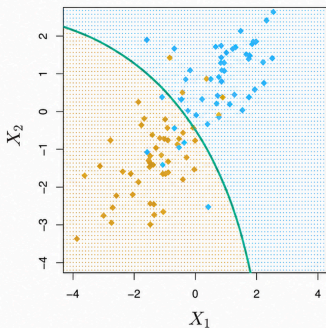


Linear Discriminant Analysis (LDA)

Quadratic Discriminant Analysis (QDA)

Quadratic Discriminant Analysis extends LDA by allowing **class-specific covariance matrices**.

- Each class has its **own** covariance matrix Σ_j
- This leads to **quadratic** decision boundaries
- More parameters than LDA: lower bias but higher variance



Discriminant Analysis in R: lda() and qda()

Linear and Quadratic Discriminant Analysis are implemented in the MASS package.

- lda() fits a Linear Discriminant Analysis model
- qda() fits a Quadratic Discriminant Analysis model

Example: LDA workflow in R (the same structure applies to QDA)

```
# Fit the LDA model
fit_lda <- lda(formula, data = <data>)

# Predict on new data
pred_lda <- predict(fit_lda, newdata = <data>)

# Posterior class probabilities
prob_lda <- pred_lda$posterior

# Predicted class labels
class_lda <- pred_lda$class
```

Example: LDA on the Pima data

```
# Fit LDA model on the training data  
pima_lda <- lda(type ~ ., data = Pima.tr)
```

```
# Estimated prior probabilities (class proportions)  
pima_lda$prior %>% round(3)
```

```
      No   Yes  
0.66 0.34
```

```
# Class-specific mean vectors (sample means within each class)  
pima_lda$means %>% round(1)
```

	npreg	glu	bp	skin	bmi	ped	age
No	2.9	113.1	69.5	27.2	31.1	0.4	29.2
Yes	4.8	145.1	74.6	33.1	34.7	0.5	37.7

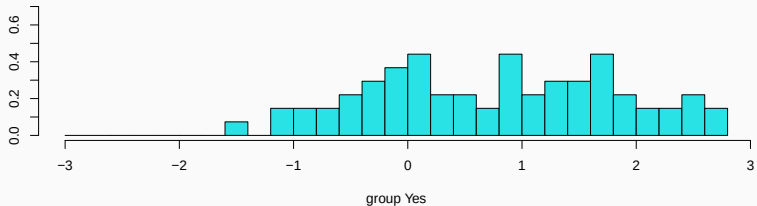
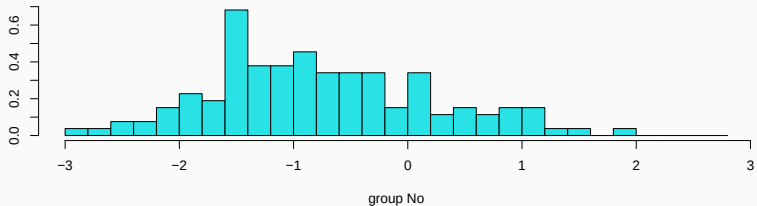
Linear discriminant functions and projections

```
# Coefficients (weights) defining the discriminant directions  
pima_lda$scaling %>% round(4) %>% t()
```

```
# Projection of observations onto LD1  
plot(pima_lda)
```

Linear discriminant functions and projections

	npreg	glu	bp	skin	bmi	ped	age
LD1	0.0795	0.024	-0.0018	-8e-04	0.0495	1.2531	0.0314



Linear discriminant functions and projections

$$LD1 = 0.0795(npreg) + 0.024(glu) - 0.0018(bp) - 0.0008(skin) + 0.0495(bmi) + 1.2531(ped) + 0.0314(age)$$

- `scaling` contains the coefficients (weights) used to form the LDs
- The plot shows observations projected onto these LD axes

Interpretation: LD1 projection

- The histograms show the projection of the observations onto the LD1, separately for each class.
- Observations in the No group tend to have lower (more negative) LD1 values
- Observations in the Yes group tend to have higher (more positive) LD1 values
- There is some overlap between the two distributions, indicating observations that are harder to classify.

Classification is based on where an observation falls along the LD1 axis.

->

->

->

Predictions with LDA

```
# Predict on the test data
```

```
pima_pred <- predict(pima_lda, newdata = Pima.te)
```

```
# First 10 predictions
```

```
data.frame(  
  posterior = round(pima_pred$posterior, 3), # P(Y = j | X)  
  class      = pima_pred$class,             # predicted class  
  observed   = Pima.te$type                 # true class  
)[1:10, ]
```

	posterior.No	posterior.Yes	class	observed
1	0.198	0.802	Yes	Yes
2	0.969	0.031	No	No
3	0.982	0.018	No	No
4	0.971	0.029	No	Yes
5	0.153	0.847	Yes	Yes
6	0.208	0.792	Yes	Yes
7	0.647	0.353	No	Yes
8	0.841	0.159	No	No
9	0.585	0.415	No	No
10	0.796	0.204	No	Yes

Pros and cons of discriminant analysis

Pros

- Can perform better than logistic regression when model assumptions are approximately satisfied
- Uses class structure (means and covariances) explicitly for classification

Cons

- LDA assumes normally distributed predictors with a common covariance matrix across classes
- QDA relaxes the equal-covariance assumption, but requires estimating more parameters
- QDA therefore has higher variance and needs more data

How to evaluate classification models

Classification models are usually evaluated using tools designed for class labels.

- **Deviance and AIC:** assess model fit and help compare models. AIC also accounts for model complexity. For both, lower values generally indicate a better model.
- **Confusion matrix:** compares observed and predicted classes, showing correct and incorrect classifications
- **Accuracy, sensitivity, and specificity:** summarize different aspects of classification performance
- **ROC curve and AUC:** evaluate how well the model separates the classes across different classification thresholds

Deviance and AIC for a simple model: Pima data

Model with a single predictor (glu)

```
pima_glm1 <- glm(type ~ glu,family = binomial, data = Pima.tr); summary(pima_glm1)
```

Call:

```
glm(formula = type ~ glu, family = binomial, data = Pima.tr)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-5.503636	0.836077	-6.583	4.62e-11 ***
glu	0.037784	0.006278	6.019	1.76e-09 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 256.41 on 199 degrees of freedom

Residual deviance: 207.37 on 198 degrees of freedom

AIC: 211.37

Number of Fisher Scoring iterations: 4

Residual deviance is 207 and AIC 211. These values are meaningful **only when compared** to those

of other models

Deviance and AIC for more complex model with all predictors

Residual deviance dropped with 29, and AIC is lower

```
pima_glm_full <- glm(type ~ ., family = binomial, Pima.tr); summary(pima_glm_full)
```

Call:

```
glm(formula = type ~ ., family = binomial, data = Pima.tr)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-9.773062	1.770386	-5.520	3.38e-08	***
npreg	0.103183	0.064694	1.595	0.11073	
glu	0.032117	0.006787	4.732	2.22e-06	***
bp	-0.004768	0.018541	-0.257	0.79707	
skin	-0.001917	0.022500	-0.085	0.93211	
bmi	0.083624	0.042827	1.953	0.05087	.
ped	1.820410	0.665514	2.735	0.00623	**
age	0.041184	0.022091	1.864	0.06228	.

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 256.41 on 199 degrees of freedom
Residual deviance: 178.39 on 192 degrees of freedom
AIC: 194.39

Confusion matrix

A confusion matrix is a cross-tabulation of **observed** and **predicted** classes.

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

- Accuracy (proportion of correctly classified observations):
 $(TP + TN)/(TP + TN + FP + FN)$
- Sensitivity (True Positive Rate: ability to detect positives): $TP/(TP + FN)$
- Specificity (True Negative Rate: ability to detect negatives): $TN/(TN + FP)$

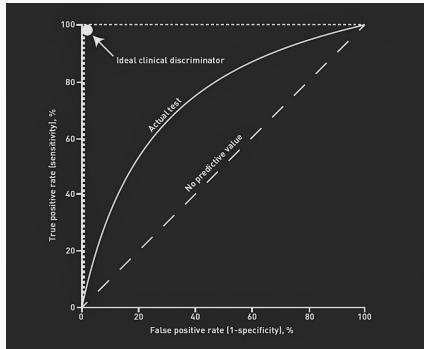
ROC curve and AUC

ROC (Receiver Operating Characteristic) curve

- Plots the **True Positive Rate (Sensitivity)** against the **False Positive Rate (1 – Specificity)** for all possible classification thresholds. It is designed to show the trade-off between catching true positives and making false alarms.

AUC (Area Under the Curve)

Summarizes the ROC curve into a single number: E.g. $AUC = 0.5$ (no better than random guessing) and $AUC = 1.0$ (perfect)



Typical interpretation of AUROC

AUROC	Interpretation
0.5	Random guessing
0.6–0.7	Weak discrimination
0.7–0.8	Acceptable
0.8–0.9	Good
0.9–1.0	Excellent

Model comparisons

Comparing GLM, LDA, and QDA: no CV, one split, and CV

Comparing the classification performance using Pima data.:

Without cross-validation

- Fit each model on the full dataset
- Evaluate performance on the same dataset
- Compare accuracy, confusion matrix, and ROC/AUC
- May give overly optimistic performance estimates

One train-test split

- Split the data into training and test sets
- Fit models on the training set
- Evaluate once on the held-out test set
- Compare accuracy and confusion matrix using class predictions
- Compare ROC/AUC using predicted probabilities or scores
- Results depend on the particular split

With cross-validation

- Split the data into several folds
- Fit models repeatedly using different training folds
- Evaluate on held-out validation folds
- Compare cross-validated accuracy, confusion matrix, and ROC/AUC
- Gives a more stable estimate of model performance

Confusion matrices (no CV)

- GLM and LDA give identical results, indicating approximately linear class separation
- QDA identifies more true positives and predicts diabetes better than GLM/LDA
- QDA also produces more false positives, misclassifying more non-diabetic cases

```
      glm
observed  No  Yes
      No  317  38
      Yes   75 102
```

```
      lda
observed  No  Yes
      No  317  38
      Yes   75 102
```

```
      qda
observed  No  Yes
      No  305  50
      Yes   69 108
```

Cross-validated accuracy using caret

Comparing Logistic Regression, LDA, and QDA using 5-fold cross-validation.

```
Pima$type <- factor(Pima$type)

set.seed(111)
# Cross-validation setup
ctrl <- trainControl(method = "cv", number = 5)
# Logistic Regression
BLR <- train(type ~ ., data = Pima,
              method = "glm", family = binomial,
              metric = "Accuracy",
              trControl = ctrl)
# Linear Discriminant Analysis
LDA <- train(type ~ .,
              data = Pima, method = "lda",
              metric = "Accuracy", trControl = ctrl)

# Quadratic Discriminant Analysis
QDA <- train(type ~ .,
              data = Pima, method = "qda",
              metric = "Accuracy",
              trControl = ctrl)
```

Cross-validated accuracy using caret

```
[[1]]
```

```
      GLM
```

```
Observed  No  Yes  
      No  314  41  
      Yes   76 101
```

```
$Accuracy
```

```
[1] 0.7800752
```

```
[[1]]
```

```
      LDA
```

```
Observed  No  Yes  
      No  315  40  
      Yes   75 102
```

```
$Accuracy
```

```
[1] 0.7838346
```

```
[[1]]
```

```
      QDA
```

```
Observed  No  Yes  
      No  301  54  
      Yes   77 100
```

```
$Accuracy
```

```
[1] 0.7537594
```

Accuracy with and without cross-validation:

- LDA achieves the highest accuracy under cross-validation
- GLM performs very similarly to LDA, but slightly worse under cross-validation
- QDA performs worse than GLM and LDA, and shows reduced performance under cross-validation, indicating higher variance

	CV	No CV
GLM	0.780	0.788
LDA	0.784	0.788
QDA	0.754	0.776

- Note: Choose models based on cross-validation, because it reflects performance on new data.

Test-set performance using one train-test split

The models are trained on `Pima.tr` and evaluated once on the held-out test set `Pima.te`.

GLM confusion matrix

GLM

Observation	No	Yes
-------------	----	-----

No	200	23
----	-----	----

Yes	43	66
-----	----	----

Accuracy: 0.8012

LDA confusion matrix

LDA

Observation	No	Yes
-------------	----	-----

No	198	25
----	-----	----

Yes	42	67
-----	----	----

Accuracy: 0.7982

QDA confusion matrix

QDA

Observation	No	Yes
-------------	----	-----

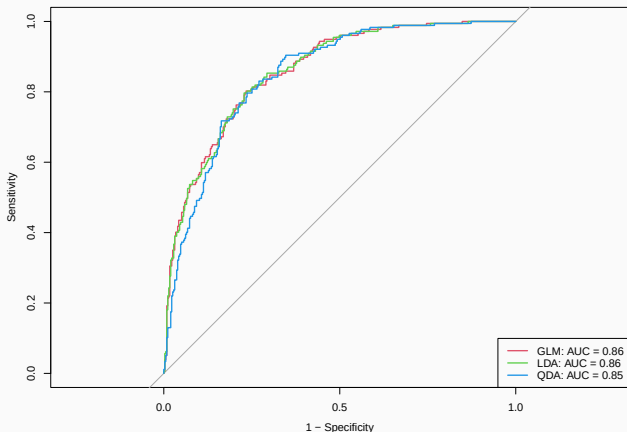
No	194	29
----	-----	----

Yes	47	62
-----	----	----

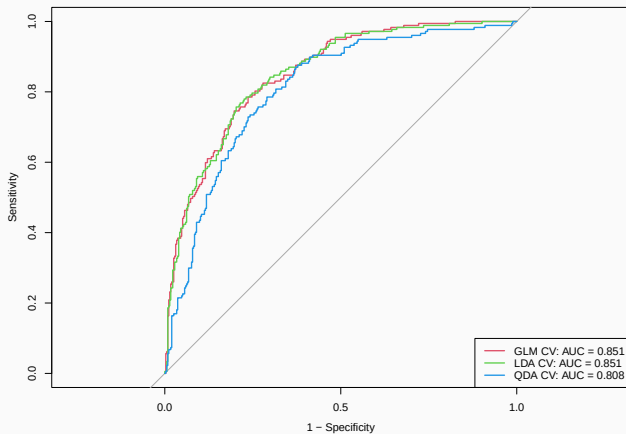
Accuracy: 0.7711

ROC/AUC on the full dataset (no CV): GLM, LDA, and QDA

- ROC curves compare classifiers across all possible classification thresholds.
- GLM and LDA have nearly identical ROC curves and same AUC values
- QDA has a slightly lower AUC
- Differences are small, indicating similar discriminative ability



Cross-validated ROC/AUC: GLM, LDA, and QDA



Take-home messages

- Classification predicts **class membership** for categorical outcomes
- Logistic regression models probabilities directly and provides interpretable coefficients
- LDA and QDA classify by modeling class distributions
- Different classifiers should be compared using suitable criteria: confusion matrices, ROC/AUC, deviance, and AIC
- Cross-validation is essential for judging how well a model generalizes to new data

Remark 1: choose the classifier that gives the best balance between interpretability, assumptions, and predictive performance.

Remark 2: logistic regression is often a strong and interpretable baseline, while LDA/QDA can perform well when their assumptions are approximately satisfied.